POC Brief: Instagram Catalog Intelligence Pipeline

Type: Research POC (throwaway code, keepable learnings) **Owner:** AI Intern · **Sponsor:** Ayodele **Feeds into:** Vendor onboarding flow (paste/connect handle → parsed store → "needs attention" review) and, later, the DM sales agent knowledge base (PRD-12 AI layer) **Status:** Final — intern handoff **Test accounts:** Provided by Ayodele (3–5 Professional accounts, added as Meta app testers) **Budget:** $50 hard cap on hosted inference, tracked in the cost log from day one

1. Context

When a vendor connects their Instagram account, we pull their posts (images, captions, comments, timestamps) and must turn that raw feed into a structured, sellable catalog — plus a short list of things that need the vendor's attention (missing prices, ambiguous items, suspected out-of-stock). This POC explores how to do that with a **multi-stage AI pipeline** where each stage uses the cheapest model that can do that job well, instead of one big expensive model call per post.

The production AI layer runs GPT-4o Mini on our NestJS stack. **GPT-4o Mini is therefore the baseline to beat**: for each stage, an open-source or cheaper hosted model wins only if it matches quality at lower cost, or beats quality at equal cost.

2. What the POC must prove

1. A cascade pipeline (cheap model first, expensive model only on escalation) can parse a real vendor account at high accuracy and **very low cost per account**.
2. Nigerian Instagram caption/comment conventions can be reliably extracted (see §8 — this is the hard, differentiating part).
3. The "needs attention" flags are trustworthy enough that a vendor reviewing them feels "it mostly worked, just confirm a few things" — not "I have to redo everything."
4. We know, with data, **which model wins each stage** and what the full parse costs.

Success criteria (targets, negotiable with evidence):

Metric	Target
Product-vs-not-product triage precision	≥ 90%
Price extraction accuracy (when a price exists in caption or image)	≥ 85%
Missing-price detection (recall — we must never silently invent a price)	100%
Stale/out-of-stock flag precision	≥ 75% (it's a *flag*, vendor confirms)
Carousel classification (gallery vs. multi-product) accuracy	≥ 85%
Full parse cost, 100-post account	≤ $0.15, stretch ≤ $0.05
Full parse wall-clock, 100-post account	≤ 10 minutes
Sync: change-classification precision (two-snapshot test)	≥ 85%
Sync cost on an unchanged account	≈ $0 (zero AI calls)

3. Non-goals

- Not production code. Standalone Python repo, **not** in the monorepo. What we keep: prompts, eval harness + golden set, cost data, findings.
- No fine-tuning in this phase. Prompting + routing + cheap models first; fine-tuning is a follow-up experiment only if prompting plateaus.
- No DM automation, no storefront rendering, no rules engine. Input = connected account; output = `catalog.json` + `attention.json`.
- No scraping. Official Meta API only (§4). This is a compliance line, not a preference.

4. Ingestion & compliance (Stage 0 — no AI)

- Register a Meta developer app; add the **Instagram API with Instagram Login** product. Scope: `instagram_business_basic` (read profile + media).
- The test vendor account must be a **Professional (Business/Creator)** account — personal accounts have no API access.
- **Live testing is legal without App Review**: in development mode the app supports up to ~25 test users. Add our own/friendly vendor accounts as testers. This covers the whole POC (and the concierge cohort later).
- Pull per account: profile (bio, name, category), media list (media URL, caption, timestamp, media type, permalink), and comments per post (paginate).
- **Carousels: always cache every slide.** For carousel albums, pull the `children` edge and store all child media IDs + images — even though the POC doesn't process them (§5 Stage 3). Storage is cheap; historical re-pulls are often impossible. This is insurance for full carousel extraction later.
- Respect rate limits (~200 calls/user/hour on business use cases). Cache raw pulls to disk (JSON dump per account) so the pipeline can be re-run offline without re-hitting the API — this matters for cheap iteration.
- Comments contain real people's handles/text — treat dumps as sensitive, don't commit them to the repo, and strip commenter identities from anything that leaves the machine.

5. Pipeline architecture — the funnel

```
Stage 0  INGEST        API pull → raw JSON dump (no AI)
Stage 1  ACCOUNT       What kind of business is this? (1 call per account)
Stage 2  TRIAGE        Product post vs. not? (cheap text → vision escalation)
Stage 3  EXTRACT       Structured product: name, variants, price, qty (product posts only)
Stage 4  SIGNALS       Stale? Out of stock? Inventory hints? (comments + age)
Stage 5  FLAGS         Confidence routing → auto-import vs. "needs attention"
Stage 6  SYNC          Re-pull → diff → AI on deltas only (living catalog)
```

Design rule for every stage: **cheapest thing first, escalate on low confidence.** The funnel is the cost strategy — most posts should never touch a vision model or a frontier model.

Stage 1 — Account profiling (1 call per account)

Input: bio + a sample of ~10 recent captions + posting frequency. Output (JSON): business category (fashion / food / gadgets / beauty / services / mixed), seller style (catalog poster vs. lifestyle mixer), language mix (English / Pidgin / Yoruba / mixed), typical pricing behavior (prices in captions? "DM for price"? prices on images?). Why it matters: this profile becomes **context injected into every downstream prompt** — a food vendor's "portions left" and a sneaker vendor's "sizes 40–45" need different extraction behavior. This is the "understand the profile first" step.

Stage 2 — Post triage (highest volume, must be cheapest)

Question per post: `product_listing | announcement | testimonial/repost | meme/personal | ad_creative`, plus a confidence score.

- **Pass A (text-only, small model):** caption + Stage 1 profile. Expect this to confidently handle the majority.
- **Pass B (vision, escalation only):** if Pass A confidence < threshold or caption is empty/emoji-only, send the image to a small vision model. Track the escalation rate — it drives cost. If >40% of posts escalate, the text prompt needs work before reaching for bigger models.

Stage 3 — Product extraction (product posts only)

Output a strict schema (Pydantic-validated; reject and retry on invalid JSON):

```
        {
          "name": "...", "description": "...",
          "images": ["hero_media_url", "angle_2_url", "angle_3_url"],
          "variants": [{"type": "color|size|style", "values": ["..."]}],
          "price": {"value": 25000, "currency": "NGN", "source": "caption|image|comment|none", "confidence":
0.0},
          "quantity_signal": {"kind": "explicit_count|low_stock|restock|none", "evidence": "..."},
          "negotiation_signal": "negotiable|fixed|unknown",
          "extraction_confidence": 0.0
```

```
        }
```

- **Price-on-image is a first-class path**, not an edge case — many vendors overlay the price on the photo. Experiment axis: classic OCR first (PaddleOCR/Tesseract) with an LLM interpreting the OCR text, vs. sending the image straight to a cheap vision model. Measure both on cost and accuracy.
- `source: "none"` is a **valid and important** output. The pipeline must never guess a price. Missing price → Stage 5 flag where the vendor chooses fixed vs. negotiable.
- **Carousels split into three cases**, in descending frequency: **(1) gallery** — one product, many angles: the common case. Classifier says single-product → all cached slides auto-attach as the ordered `images` array (slide 1 = hero, order preserved — vendors sequence photos deliberately), extraction runs once on caption + hero. No flag, near-zero extra cost. **(2) colorways** — one product, each slide a variant: treat as gallery in the POC; slide-to-variant mapping is a stretch refinement. **(3) true multi-product** — distinct items per slide: the only case that flags ("one item or three colorways?"). **Flag resolution UX:** two choices — "one product" (collapse to gallery) or "split", where each slide pre-seeds a draft product with its image attached and the vendor manually fills name, price, and quantity per slide. Manual inventory entry stays in the vendor's hands; the pre-seeded slides keep it a 30-second task. Full multi-product extraction (per-slide vision, cross-slide grouping, caption-to-slide price alignment) is stretch, not core. So the carousel classifier's core question is binary: *one product or several?* **Complexity valve:** if the classifier proves finicky or time runs short, the sanctioned fallback is no classifier at all — every carousel attaches all slides as one product's gallery (config flag, zero AI cost; rare multi-product carousels render as one messy product, which vendors can fix by hand). Future-proofing rules so multi-product stays additive later: (a) post→products is **1:N in the data model from day one**; (b) carousel children are always cached at ingest (§4); (c) vendor resolutions of multi-item flags are stored verbatim — they are the hand-labeled dataset for the future extraction work. **Promotion trigger:** if week-1 golden-set labeling shows a large share of *true multi-product* carousels (>⅓ of product posts), flag it to Ayodele immediately — extraction gets promoted from stretch to core.

Stage 4 — Signals (staleness / stock)

Inputs: post age, comment thread, caption edits if visible. Heuristics + LLM on the comment thread:

- "Still available?" with no vendor reply after N days → stale suspicion
- Vendor replies "sold", "gone", "out of stock", "sold out" → out-of-stock signal (quote the comment as evidence)
- "Restocked", "back in stock" in newer posts referencing the same item → refresh signal
- Post older than X months with zero recent engagement → low-confidence stale flag Cheap trick to test first: a keyword/regex prefilter over comments so the LLM only reads threads that contain candidate signals.

Stage 5 — Flags & routing

Every extracted product lands in one of three buckets by confidence:

1. **Auto-import** (high confidence, price found or explicitly negotiable)
2. **Needs attention** (missing price, ambiguous variants, stale suspicion, "is this a product?" uncertainty) — each flag carries the post thumbnail, the question, and one-tap answer options (this maps 1:1 to onboarding screen A3)
3. **Auto-exclude** (memes, reposts, announcements) — but list them so the vendor can rescue mistakes The flag list is the product. A vendor with 100 posts should see ≤ 10 attention items on a typical account.

Stage 6 — Sync (the living catalog)

After first import, the catalog stays in sync with Instagram: auto-sync on a schedule (vendor-toggleable) plus a manual "Sync now" that is always available.

Trigger reality: Meta provides **no webhooks for media** (create/edit/delete) — auto-sync is scheduled polling, frequency tiered by vendor activity. **Comments DO have webhooks**, so stock signals ("sold" replies, unanswered "still available?") can arrive near-real-time without polling.

Diff before AI — sync must cost ~nothing when nothing changed:

1. **Anchor on media ID** (stable). New ID → full pipeline (Stages 2–5). ID gone → vendor deleted the post → propose `archived` with a "still selling this?" flag; never silently delete.

2. **Content hash** per post (caption + media URL + comment count). Hash unchanged → hard no-op, zero AI calls.
3. **Caption changed** → re-run Stage 3 on that post only; diff structured output against the stored product. Price change → update + **attention flag if the new price crosses the vendor's negotiation floor** (rules-engine consistency check).
4. **New comments only** (store a per-post comment cursor) → Stage 4 signals on the delta. "Sold" / "it don finish" → propose `out_of_stock`.
5. **Repost dedup (semantic):** vendors repost the same item as a *new* post (new media ID, same product). Embed new products and compare against the catalog (any vector lib for the POC; pgvector in prod). High similarity → same item: merge and treat as a refresh/restock signal, never duplicate.

Product lifecycle state machine: `active → low_stock → out_of_stock → stale → archived` (transitions in both directions). Every sync event *proposes* a transition.

Auto vs. ask — decided policy: the default is **ask-first at almost every decision**. Sync *proposes* changes; the vendor confirms. Only true no-brainers self-apply out of the box (e.g., merging an exact repost of an already-imported item). Everything else — out-of-stock transitions, archiving deleted posts, importing new products, and especially price changes — prompts the vendor. To reduce prompting over time, vendors grant **granular permissions per change type** ("auto-mark items out of stock", "auto-import new posts", "auto-archive deleted posts"); each permission is opt-in, revocable, and scoped to exactly one change type. Price changes always ask, permission or not — they touch the negotiation rules and real money. Every auto-applied change carries its evidence (quoted comment / caption diff), is logged, and is one-tap reversible; the vendor receives a WhatsApp digest, not a firehose: "Sync found 4 changes — 1 applied, 3 need you."

POC scope: a `sync <handle>` command that re-pulls, diffs against the cached previous run, and emits `changes.json` — no-ops counted, deltas classified, proposed state transitions with evidence attached.

6. Model matrix (starting candidates — intern updates with current options)

Stage	Open-source candidates	Cheap hosted candidates	Baseline
1 Account profile	Llama 3.x 8B, Qwen 2.5 7B	Gemini Flash-Lite, Claude Haiku	GPT-4o Mini
2 Triage (text)	Qwen 2.5 7B, Llama 3.x 8B, even a fine-tune-free SetFit/embedding classifier	Gemini Flash-Lite	GPT-4o Mini
2 Triage (vision escalation)	Qwen 2.5-VL 7B, Llama 3.2 Vision 11B	Gemini Flash (vision is very cheap)	GPT-4o Mini vision
3 Extraction	Qwen 2.5 7B/14B + strict JSON	Gemini Flash, Claude Haiku	GPT-4o Mini
3 Price OCR	PaddleOCR / Tesseract (+ small LLM to interpret)	vision model direct	—
4 Signals	regex prefilter + small model	Gemini Flash-Lite	GPT-4o Mini

- Self-hosting for dev: **Ollama** locally is free for iteration; for realistic cost/latency numbers use hosted open-model inference (Groq, Together, DeepInfra, Fireworks), ideally routed through **OpenRouter** so every model — open or proprietary — sits behind one API key and one model string.
- Model names above are starting points from early 2026 — the intern's first task includes refreshing this table with whatever is current and cheapest per capability class.

7. Cost engineering rules

1. **Cascade everything.** No post touches a vision or frontier model unless a cheaper pass failed or lacked confidence.
2. **Log cost per stage per post** (tokens in/out × price). The final report must show a cost breakdown pie for a real account — this tells us where to optimize next.
3. **Cache aggressively.** Raw API dumps, image downloads, OCR output, and stage outputs are all cached; re-runs only re-execute changed stages.
4. **Batch where the provider supports it** (offline parsing is not latency-sensitive; batch APIs are typically ~50% cheaper).
5. **Short prompts win.** The Stage 1 profile lets downstream prompts stay compact instead of re-

explaining the account every call.

8. Evaluation — golden set FIRST

Before building any pipeline stage, hand-label a golden set. This is the discipline that makes the whole experiment mean something; without it, everything "looks good" and nothing is known.

- 3–5 real vendor accounts across categories (fashion, food, gadgets at minimum), 50–100 posts total.
- Label per post: post type, product name(s), variants, price (value + where it appears), quantity/stock signals, staleness verdict, expected flag (if any).
- Build the eval harness (a script that runs the pipeline against the golden set and prints per-stage precision/recall + cost) **before** tuning prompts. Every prompt/model change gets a score, not a vibe.

Nigerian conventions the golden set must cover (this list is the moat — extend it):

- Prices written as `25k`, `₦25,000`, `#25k` (naira typed as `#`), `25,000 only`
- `DM for price`, `price in bio`, `send a DM 📩`
- Price overlaid on the image, not in the caption
- Sizes as ranges: `40–45`, `S–XXL`; colors listed with emojis
- Pidgin/Yoruba-mixed captions and comments
- `sold` / `gone` / `it don finish` in comment replies
- WhatsApp numbers in bio/captions (capture — useful later, but treat as PII)
- Carousel posts where each slide is a different item

9. Tooling recommendations

- **Orchestration:** LangGraph (LangChain's graph runtime) fits the funnel shape — stages as nodes, confidence-based conditional edges. Acceptable lighter alternative: plain Python + **LiteLLM** (one interface over every provider, per-call cost logging built in) + **Instructor/Pydantic** for structured outputs. Intern's choice; justify it in the findings. Keep it simple — the pipeline is the experiment, not the framework.
- **Hard requirement — models are config, not code:** every stage reads its model from a config file (per-stage model registry). Switching any stage to any model, open-source or API, must be a one-line config change with zero code changes — route calls through OpenRouter or LiteLLM to make this true. The eval harness should accept a config, so comparing models is: swap config, run harness, compare scores.
- **Repo shape:** `ingest/`, `pipeline/stages/`, `eval/golden/`, `eval/harness.py`, `runs/` (cached outputs), `report/`.
- **Demo surface:** a CLI (`parse-account <handle>`) producing `catalog.json` + `attention.json`, plus (optional, stretch) a single-page local viewer that renders the attention flags like onboarding screen A3.

10. Milestones (~4 weeks)

Week	Milestone	Done means
1	Meta dev app live, test account connected, raw dumps cached; **golden set labeled**; model matrix refreshed	We can re-run offline; eval harness scores a dummy pipeline
2	Stage 1 + 2 working with cascade; eval scores on triage	Escalation rate and triage precision measured across ≥3 models
3	Stage 3 + 4: extraction, price OCR path comparison, signals	Extraction accuracy + price accuracy measured; cost per account known
4	Stage 5 flags, end-to-end live run on a real account, findings report	Demo: connect account → catalog + attention list; one-page findings: winning model per stage, cost breakdown, failure gallery
Stretch (wk 5)	Stage 6 sync: take a second snapshot of one account ~a week after the first, build the diff engine, emit `changes.json`	Change-detection precision measured against the hand-labeled two-snapshot pair; unchanged-account sync verified at ≈ $0

11. Guardrails & gotchas

- **Never invent a price.** `source: "none"` + flag is always correct; a hallucinated price is the one unforgivable failure mode (it becomes real money at checkout).
- **Confidence must be calibrated-ish.** If everything is 0.95, thresholds are meaningless — spot-check that low-confidence items are actually worse.
- **Meta tokens expire (~60 days)** — fine for the POC, but note refresh in findings for production.
- **Rate limits:** back off on 429s; the disk cache should make this rare after the first pull.
- **Data hygiene:** no raw dumps or golden-set images in git; commenter identities stripped from shared artifacts.
- **Keep a failure gallery.** Screenshots of the funniest/worst misparses are as valuable as the metrics — they define the next iteration.

12. Working agreements

- **Test accounts:** Ayodele provides 3–5 friendly Professional vendor accounts and adds them as Meta app testers — don't recruit your own.
- **Budget:** $50 hosted-inference cap. Reach out to Ayodele *before* exceeding it — running out silently is a process failure; needing more is not.
- **Model switching is expected.** Trying and swapping models is the point of the experiment; the config-driven registry (§9) exists to make this cheap. Record every swap and its eval score in the findings.
- **Check in at each milestone** with a short async update: what was measured, what's next, any blockers. Reach out early when changing direction — a surprise at week 4 is the only wrong move.
- **This brief is self-contained on purpose.** If reality (APIs, models, costs) contradicts something written here, flag it — the brief serves the experiment, not the other way around.